

Plant Disease Detection Project Documentation (Phases 1–5)

Project Overview

Goal: Build an AI-powered Plant Disease Detection system using the PlantVillage dataset. The system identifies plant diseases from leaf images using a deep learning model based on MobileNetV2. Current progress covers environment setup, dataset preparation, verification, model training, and image prediction.

Phase 1: Environment Setup

Tasks completed: - Installed TensorFlow and required Python libraries. - Diagnosed TensorFlow / NumPy / h5py compatibility issues. - Verified TensorFlow imports successfully. - Established a clean development environment for model training.

Phase 2: Dataset Preparation

Dataset used: PlantVillage subset. Classes included: Pepper__bell__Bacterial_spot
Pepper__bell__healthy Potato__Early_blight Potato__Late_blight Potato__healthy
Tomato_Bacterial_spot Tomato_Early_blight Tomato_Late_blight Tomato_Leaf_Mold
Tomato_Septoria_leaf_spot Tomato_Spider_mites_Two_spotted_spider_mite
Tomato__Target_Spot Tomato__Tomato_YellowLeaf__Curl_Virus
Tomato__Tomato_mosaic_virus Tomato_healthy

Phase 3: Dataset Verification

Dataset path was inspected and corrected. A nested PlantVillage directory was discovered and fixed by using the inner directory as DATASET_PATH. Final dataset statistics: - Total images: 20,638 - Classes: 15 - Training images: 16,511 - Validation images: 4,127 TensorFlow image_dataset_from_directory was used to verify loading and class detection.

Phase 4: Model Training

Architecture: - Transfer Learning with MobileNetV2 - Image size: 224 x 224 - Batch size: 32 - Validation split: 20% - Epochs: 10 Data augmentation: - Random horizontal flip - Random rotation - Random zoom Training pipeline: Input Image -> Data Augmentation -> MobileNetV2 Preprocessing -> MobileNetV2 Base Model -> GlobalAveragePooling2D -> Dropout(0.3) -> Dense Softmax Layer Outputs generated: - plant_disease_model.keras - class_names.txt

Phase 5: Prediction System

A prediction pipeline was implemented. Steps: 1. Load trained model. 2. Load class names. 3. Resize image to 224x224. 4. Convert image to array. 5. Predict disease class. 6. Display confidence score. Bug discovered: The image was preprocessed twice using MobileNetV2 preprocessing. Result: Model predicted Tomato_Late_blight with ~99% confidence for nearly every image. Root Cause: preprocess_input() was present both inside the trained model and inside predict.py. Fix: Removed preprocessing from predict.py and kept preprocessing only inside the model. Outcome: Predictions became correct and aligned with actual classes.

Validation Results

Model evaluation: - Validation Accuracy: 89.6% - Validation Loss: 0.3048 This confirmed that the model learned meaningful disease features and was functioning correctly.

Current Project Status

Completed: Phase 1 - Environment Setup Phase 2 - Dataset Preparation Phase 3 - Dataset Verification Phase 4 - MobileNetV2 Training Phase 5 - Prediction System Next: Phase 6 - Flask Web Application Phase 7 - Disease Recommendation Engine Phase 8 - AI Assistant Integration